one-debugger Basics

Starting the debugger

1. Install the tools (Debian, Ubuntu, Linux Mint):
   `sudo apt install cmake libsdl2-dev libncurses-dev`
2. Build the emulator and the debugger:
   `cmake --preset linux-debugger`
   `cmake --build --preset linux-debugger`
3. Run it: `one-debugger game.nes` (or with no game: **F3** / File > Open game, or `load game.nes`)
   The debugger stops at the first instruction after RESET. The game runs in its own window; the debugger uses the terminal.
4. Press **F5** to run, **Ctrl+D** to stop again.

Debugging your own program

1. Link with debug info: `ld65 ... --dbgfile game.dbg`
2. Load the labels: `one-debugger -l game.dbg game.nes`
   Labels show up in the code window and work wherever an address does: `bpx nmi`, `g main`. `lsave` saves them, with yours from `n`, in `game.mlb`, which loads by itself.
3. Put commands you always type in `game.init` next to the ROM: it runs at start-up (or `-s setup.txt`).
4. Pick the colors: `--theme borland`, or a config file (`one-debugger --write-config FILE`).

Essential Commands

Type commands in the command window; they are not case sensitive. Addresses are hexadecimal, with or without `$`, or a label. While the game runs, **Ctrl+D** (or **F5**) breaks back into the debugger.

X — Execute: run until a breakpoint is hit. `x`
Func key: [F5]. [Ctrl+D] stops.

T — Trace (step into) instructions. `t [count]`
Func key: [F8]

P — Program step: steps over subroutines. `p [ret]`
Func key: [F10], [F12] for p ret
p ret: go until the current subroutine returns (RTS or RTI) and stop in the caller

G — Go. `g [address]`
Runs until the address becomes the Program Counter, or a breakpoint is hit.
Without an address: run until a breakpoint is hit. [Ctrl+D] stops.

BPX — Breakpoint on execution. `bpx [bank:]address [if condition] [do "commands"]`
Example: bpx $8432 or bpx nmi_handler or bpx 8000 if a == 0 do "d 300; g"
bank: (hex, as the code window shows it) stops only while that PRG bank is mapped there. Example: bpx 05:BAB7
if: Lua (a x y s p pc scanline cycle frame mem[addr] emu.*). do: commands when it stops
Func key: [F9] toggles a breakpoint on the code window's cursor line (in a bank-switched game, in the bank mapped there now)

BL — List breakpoints. `bl`

BC — Clear breakpoints. `bc number... | *`
Example: bc 3

ONE one-debugger
Quick Reference



1. Registers window
2. PPU window
3. CPU Memory window
4. PPU Memory window
5. Code window
6. Command window
7. OAM window
8. Watch window

A. The current instruction (PC), with what it will do
B. A breakpoint (F9 toggles one at the cursor)
C. A label from `-l` or `n`
D. Values that changed since the last stop
E. Status line: the function keys

Config Files

The debugger's settings (color theme, colors, window shadows): `--config FILE`, else `./one-debugger.cfg`, else `~/.config/one/debugger.cfg`. The emulator's settings (controllers, sound, hotkeys): `--one-config FILE`, else `./one.cfg`, else `~/.config/one/one.cfg`. `--write-config FILE` writes every debugger setting with its default and a description.

ONE NES emulator — generated from the debugger source by `docs/quickref/build.sh`, 2026-11-02 (git 7f56625)

one-debugger Commands

Complete list. In the debugger, `help` lists the commands and `help command` explains one.

Flow Control

T	F8	Trace (step into) instructions `t [count]`
P	F10	Program step: steps over subroutines `p [ret]`
G, GO		Go `g [address]`
X	F5	Execute: run until a breakpoint is hit `x`
HERE	F7	Go to the code window's cursor line `here`
GSL		Go to a scanline `gsl line`
FS		Frame step: run frames `fs [count]`
PRESS		Press controller buttons `press [p2] buttons [frames]`
ZAPPER		Aim the Zapper (light gun) on port 2 `zapper [x y | off] [fire [frames]] | none`
SAVESTATE		Save or load the game `savestate save|load [slot]`

Breakpoints

BPX	F9	Breakpoint on execution `bpx [bank:]address [if condition] [do "commands"]`
BPM		Breakpoint on memory access `bpm [bank:]address[..end] [r|w|rw|x] [if condition] [do "commands"]`
BOP		Breakpoint on opcode execution `bop opcode [if condition] [do "commands"]`
BC		Clear breakpoints `bc number... | *`
BD		Disable breakpoints `bd number... | *`
BE		Enable breakpoints `be number... | *`
BL		List breakpoints `bl`

Display/Change Memory

D	Display CPU memory `d [address]`
DP	Display PPU memory `dp [address]`
U	Unassemble: show code at an address `u [address]`
EB	Edit bytes `eb address = value [value...]`
EO	Edit OAM (sprite memory) `eo offset = value [value...]`
R	Change a register `r register = value`
N	Name (label) an address `n [bank:]address [name]`
WATCH	Watch memory in the watch window (shown at every stop) `watch [-p] address [type] [name]`
WC	Clear watches `wc number... | *`
COMMENT	Comment an address (shown in the code window) `comment [bank:]address [text]`
LB	Load a binary file into CPU memory `lb file address`

Search and Labels

S	Search memory for data `s [-cp] [address L length data-list]`
CS	Memory search (cheat search): find a variable by how it changes `cs start [cpu|ppu|oam]`
FREEZE	Freeze a RAM byte: written back every frame `freeze [address [value]]`
UNFREEZE	Stop freezing `unfreeze address | *`
PROF	Profiler: calls and cycles per routine `prof [on|off|reset|list [sort COLUMN] [asc] [N]]`
BT	Call stack: how the CPU got here `bt [jumps]`
CDL	Code/data log (Mesen CDL format) `cdl [on|off|clear|stats|unseen [bank [bytes]]|mark|unmark address [length]|save [file]|load [file] [force]]`
LSAVE	Save the labels `lsave [file]`
LLOAD	Load labels (added to the current ones) `lload file`
GG	Game Genie codes: decode, encode, apply `gg code | gg address value [compare] | gg on code [description] | gg off code|* | gg delete code|* | gg list | gg db [word|N] | gg enable | gg disable`

Windows

EC	F6	Enter/exit the code window `ec`
WW		Open or close the watch window `ww`
WOAM		Open or close the OAM (sprites) window `woam`
OAM		Show the sprites (OAM) decoded `oam [all]`
RS	F4	Frame in progress in the game window, at every stop `rs [on|off]`
WBG		Open or close the background window `wbg [save file.png]`
WPROF		Open or close the profiler window `wprof [save file.png]`
WPAL		Open or close the palette window `wpal [save file.png]`
WPAT		Open or close the pattern table window `wpat [save file.png]`
SCREENSHOT	Shift+F12	Save the game picture as a PNG `screenshot [file]`
LAG		Lag frame counter (FCEUX's) `lag [on|off|reset]`
VERSION		Show ONE's version `version`
FPS	Shift+F10	Frames per second in the game window's title `fps [on|off]`

Mode Control

RESET	Reset the NES `reset [hard]`
Q	Quit the debugger `q`

Miscellaneous

LOG	Emulator messages: how much each part tells `log [category] [level]`
LOAD	Load a game (power it on) `load file.nes (or file.zip: its first .nes)`
SOURCE	Run debugger commands from a file `source file`
LUA	Lua scripts (Mesen 2 and FCEUX APIs) `lua [file | stop [n]]`
H, HELP, ?	Help `help [command]`

Function Keys

F3	Open game...
Alt+X	Exit
F5	Go / stop
F8	Trace into
F7	Go to cursor (here)
F10	Step over
F12	Step to return
Ctrl+D	Break
F9	Toggle breakpoint
F4	Frame in progress
Shift+F12	Screenshot
Shift+F10	FPS counter
F6	Enter/exit code window
Ctrl+F6	Next
Ctrl+F5	Zoom
Ctrl+F7	Size/move
Ctrl+=	Larger characters
Ctrl+-	Smaller characters
F1	Command help

Window Numbers

1	Registers window
2	PPU window
3	CPU Memory window
4	PPU Memory window
5	Code window
6	Command window
7	OAM window
8	Watch window

In a Code or Memory Window

F6, ec	Code window: enter / exit (SoftICE EC)
Up, Down	Move the cursor / scroll one line
PgUp, PgDn	Scroll one page
Home	Code: back to PC. Memory: to $0000
Wheel	Scroll
Dbl-click	Code: toggle a breakpoint
;	Code: comment the cursor line (edit, Enter)
Up, Down	Command window: command history
Left, Right	Command line: move the cursor (Ctrl: by word; Home, End)
Typing	Always goes to the command line
Tab	Complete a command or cs word

In the Watch Window

watch, ww	Opens when a watch is added (or ww); closes when the last goes
Up, Down	Select a watch
Del	Remove it (wc)
Enter	Edit its value (eb on the command line)
Ins	Add a watch (watch on the command line)
Ctrl+F	Freeze / unfreeze it (F)

Addresses and Numbers

C000, $C000	Addresses and values are hexadecimal
nmi_handler	A label works wherever an address does
t 10, t $10	Counts are decimal unless they start with $
gsl 241	Scanlines are decimal (0-261)
eb 300 = 1 2	'=' and ',' are optional separators

In the OAM Window

A	All 64 sprites / the visible ones
H	The bytes / decoded
Enter	Edit the sprite (eo on the command line)

Scripting

bpx a if cond	Stop when the Lua condition is true
bpx a do "cmds"	Commands run when it stops
source file	Commands from a file; game.init runs at start-up
debugger.command	Lua: run a command, returns its output

Script Commands

Only in scripts (`-s`, `--batch`); a failed `!assert` makes the exit code 1.

```
!assert reg r value
```

F1: HELP **F5: GO** **F6: EC** **F7: HERE** **F8: TRACE** **F9: BPX** **F10: STEP** **F12: RET** **Ctrl+D: BREAK** **Alt+X: EXIT**